

Question 1: What is Spring Security?

Simple Answer

Spring Security is a powerful authentication and authorization framework for Spring applications. It helps secure web applications and REST APIs by protecting resources from unauthorized access.

Detailed Explanation

Spring Security is one of the most widely used security frameworks in the Java ecosystem. It provides built-in features to secure applications without writing security logic from scratch.

It handles common security requirements such as:

- User Authentication (Who are you?)
- User Authorization (What are you allowed to access?)
- Password Encryption
- Login and Logout
- Session Management
- JWT Authentication
- OAuth2 Integration
- Protection against common attacks like CSRF

In modern Spring Boot applications, Spring Security is mainly used to secure REST APIs using JWT authentication.

Real-Time Example

Consider an online banking application.

Without Spring Security:

- Anyone can access account details.

With Spring Security:

- User must log in.
- Username and password are verified.
- JWT token is generated.
- Every API request is validated using the JWT token.

Only authenticated users can access their account.

Code Example

Add Spring Security dependency

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Interview Tip

★ Interviewers usually ask:

"Is Spring Security only for login pages?"

Answer:

"No. Spring Security can secure web applications, REST APIs, Microservices, and even OAuth2-based applications."

Key Points to Remember

-  Part of the Spring ecosystem.
-  Provides authentication and authorization.
-  Secures REST APIs.
-  Supports JWT, OAuth2, LDAP, etc.
-  Highly customizable.

Quick Revision

Spring Security is used to secure Spring applications by handling authentication and authorization.

Question 2: Why do we need Spring Security?

Simple Answer

We need Spring Security to protect our application from unauthorized access and common security threats. It ensures that only authenticated users can access protected resources.

Detailed Explanation

Every application contains sensitive data.

Examples:

- Customer information
- Banking details
- Employee records

- Orders
- Payments

Without security:

- Anyone can access APIs.
- Data can be stolen.
- Passwords may be exposed.

Spring Security solves these problems by providing:

- Authentication
- Authorization
- Password Encryption
- Session Security
- JWT Authentication
- CSRF Protection
- CORS Support

Real-Time Example

Imagine an e-commerce application.

Public APIs:

GET /products

Protected APIs:

GET /orders
POST /payment
GET /profile

Spring Security allows everyone to view products but restricts order and payment APIs to authenticated users.

Interview Tip

★ Don't answer only:

"To secure the application."

Instead explain:

"It protects APIs using authentication, authorization, encrypted passwords, JWT validation, and various security mechanisms."

Key Points to Remember

-  Protects sensitive data.
-  Prevents unauthorized access.
-  Supports secure authentication.
-  Supports role-based access.
-  Prevents common attacks.

Quick Revision

Spring Security protects applications by allowing only authorized users to access secured resources.

Question 3: Authentication vs Authorization

Simple Answer

Authentication verifies **who the user is**, while authorization determines **what the authenticated user is allowed to access**.

Detailed Explanation

These are two different security concepts.

Authentication

Authentication answers:

"Who are you?"

The user provides:

- Username
- Password

If valid,

the user becomes authenticated.

Authorization

Authorization answers:

"What are you allowed to access?"

Example:

ADMIN

- Manage Users

- Delete Products
- View Reports

USER

- View Products
- Place Orders

Even after login, users cannot access everything.

Real-Time Example

Employee Management System

User:

j o h n

Password is correct.

Authentication:

 Success

Now John tries to access:

/admin/deleteEmployee

Spring Security checks John's role.

If role is USER

Authorization:

 Access Denied

Code Example

```
.requestMatchers("/admin/**")  
.hasRole("ADMIN")
```

Interview Tip

★ Very common interview question.

Remember this sentence:

Authentication comes first. Authorization comes next.

Key Points to Remember

-  Authentication verifies identity.
-  Authorization verifies permissions.
-  Authentication happens before authorization.
-  Both are required for secure applications.

Quick Revision

Authentication = Who are you? Authorization = What can you access?

Question 4: What is SecurityContext?

Simple Answer

`SecurityContext` is an object that stores the authentication information of the currently logged-in user during a request.

Detailed Explanation

After successful authentication, Spring Security stores the user's authentication details inside the `SecurityContext`.

It contains:

- Logged-in user
- Roles
- Authorities
- Authentication status

Whenever the application needs user information, it retrieves it from the `SecurityContext`.

Real-Time Example

A user logs in successfully.

Spring Security stores:

Username : murali

Role : ADMIN

```
Authenticated : true
```

Later,

when accessing another API,

Spring Security reads these details from the `SecurityContext`.

Code Example

```
SecurityContext context =  
SecurityContextHolder.getContext();
```

Interview Tip

★ Don't confuse

`SecurityContext`

with

`SecurityContextHolder`.

The holder stores and manages the context, while the context contains the authenticated user's security information.

Key Points to Remember

-  Stores logged-in user details.
-  Created after successful authentication.
-  Used throughout the request.
-  Contains the Authentication object.

Quick Revision

`SecurityContext` stores the authentication information of the current user.

Question 5: What is Authentication?

Simple Answer

`Authentication` is an interface that represents the authenticated user. It contains user details, credentials, roles, and authentication status.

Detailed Explanation

After login,

Spring Security creates an `Authentication` object.

It contains:

- Principal (User)
- Credentials (Password)
- Authorities (Roles)
- Authenticated status

This object is stored inside the `SecurityContext`.

Real-Time Example

After login:

Username

murali

Role

ADMIN

Spring Security creates:

`Authentication`

↓

`SecurityContext`

↓

Current Request

Now every secured API can access the logged-in user's information.

Code Example


```
Authentication authentication =  
SecurityContextHolder.getContext().getAuthentication();  
  
String username =  
authentication.getName();
```

Interview Tip

★ Frequently asked:

"What is Principal?"

Answer:

Principal represents the currently authenticated user.

Key Points to Remember

-  Represents the logged-in user.
-  Stores username and roles.
-  Stored inside SecurityContext.
-  Available throughout the request.

Quick Revision

Authentication contains the authenticated user's identity and permissions.

Question 6: Explain how a request is processed in Spring Security.

Simple Answer

Every incoming request first passes through the Spring Security filter chain. The filters authenticate the user, authorize access, and then allow the request to reach the controller if permitted.

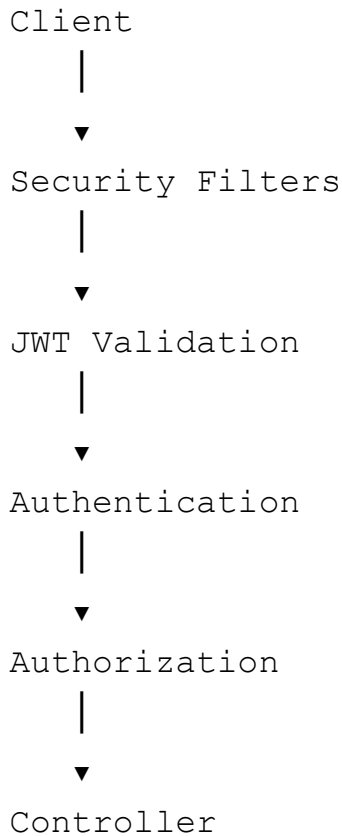
Detailed Explanation

The request processing flow is:

1. Client sends an HTTP request.
2. Request enters the Spring Security Filter Chain.
3. Authentication is performed (for example, by validating a JWT).

4. If authentication succeeds, user details are stored in the `SecurityContext`.
5. Authorization checks whether the user has permission to access the requested resource.
6. If authorized, the request reaches the controller.
7. Otherwise, Spring Security returns **401 Unauthorized** or **403 Forbidden**.

Real-Time Example



Interview Tip

★ Remember the order:

Filter Chain → Authentication → SecurityContext → Authorization → Controller

Key Points to Remember

- ✓ Every request passes through security filters.
- ✓ Authentication happens before authorization.
- ✓ Successful authentication populates the `SecurityContext`.
- ✓ Unauthorized requests are blocked before reaching the controller.

Quick Revision

Spring Security intercepts every request, authenticates the user, checks permissions, and only then allows access to the controller.

Question 7: What is the default behavior of Spring Boot Security?

Simple Answer

When the Spring Security dependency is added, Spring Boot automatically secures all endpoints. Every request requires authentication, and a default login page with a generated password is provided.

Detailed Explanation

Spring Boot follows the **secure by default** principle.

Without writing any security configuration:

- All endpoints are protected.
- A default login page is displayed for browser requests.
- A default user named `user` is created.
- A random password is generated and printed in the application logs.

This automatic setup is useful for development but is usually replaced with custom security in real projects.

Real-Time Example

You create a Spring Boot REST API:

```
GET /employees
```

Without Spring Security:

```
Accessible by everyone
```

After adding:

```
spring-boot-starter-security
```

The endpoint now requires authentication.

Interview Tip

★ Interviewers often ask:

"Where can you find the generated password?"

Answer:

"It is printed in the application startup logs."

Key Points to Remember

- ☒ All endpoints are secured by default.
- ☒ Default username is `user`.
- ☒ Random password is generated.
- ☒ Default login page is available.

Quick Revision

Spring Boot automatically protects all endpoints when Spring Security is added.

Question 8: What is SecurityFilterChain?

Simple Answer

`SecurityFilterChain` is the configuration that defines how Spring Security processes incoming requests, including authentication, authorization, and other security rules.

Detailed Explanation

Spring Security works by passing every request through a chain of security filters.

In Spring Security 6, we configure these filters using a `SecurityFilterChain` bean instead of the deprecated `WebSecurityConfigurerAdapter`.

With `SecurityFilterChain`, we can:

- Define public and protected endpoints.
- Configure authentication rules.
- Disable CSRF for REST APIs.
- Enable stateless sessions for JWT.

Code Example

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity
http)
    throws Exception {

    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated());

    return http.build();
}

```

Interview Tip

★ In modern Spring Boot (3.x), always use `SecurityFilterChain`. Avoid using `WebSecurityConfigurerAdapter` in new projects.

Key Points to Remember

-  Configures Spring Security.
-  Defines endpoint access rules.
-  Replaces `WebSecurityConfigurerAdapter`.
-  Required in Spring Security 6.

Quick Revision

`SecurityFilterChain` defines how Spring Security protects and processes requests.

Question 9: Why was `WebSecurityConfigurerAdapter` deprecated?

Simple Answer

`WebSecurityConfigurerAdapter` was deprecated to encourage a simpler, more flexible, and component-based configuration using `SecurityFilterChain` beans.

Detailed Explanation

Older versions of Spring Security required developers to extend the `WebSecurityConfigurerAdapter` class.

Example:

```
public class SecurityConfig extends  
WebSecurityConfigurerAdapter
```

From Spring Security 5.7 onward, this approach was deprecated because:

- It relied on inheritance.
- Configuration became less flexible.
- Bean-based configuration is easier to understand and test.

The recommended approach is to define a `SecurityFilterChain` bean.

Real-Time Example

Old Approach

```
extends WebSecurityConfigurerAdapter
```

Modern Approach

```
@Bean  
SecurityFilterChain
```

Interview Tip

★ If asked, don't say it was removed because it was "bad." Explain that Spring adopted a **component-based configuration model**, which is more flexible and aligns with modern Spring development.

Key Points to Remember

-  Deprecated from Spring Security 5.7.
-  Use `SecurityFilterChain` instead.
-  Encourages bean-based configuration.
-  Preferred for Spring Boot 3 applications.

Quick Revision

`WebSecurityConfigurerAdapter` was deprecated in favor of the more flexible **`SecurityFilterChain`** configuration.

Question 10: What is HttpSecurity?

Simple Answer

`HttpSecurity` is a configuration object used to define security rules for HTTP requests, such as authentication, authorization, CSRF, CORS, and session management.

Detailed Explanation

`HttpSecurity` is the main API used to configure Spring Security.

Using it, we can:

- Protect endpoints.
- Allow public URLs.
- Configure login.
- Disable CSRF for REST APIs.
- Configure session management.
- Add custom filters like JWT filters.

It is commonly used inside the `SecurityFilterChain` bean.

Code Example

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity
http)
    throws Exception {

    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/login").permitAll()
            .anyRequest().authenticated());

    return http.build();
}
```

Real-Time Example

A company wants:

- `/login` → Public

- `/register` → Public
- `/employees/**` → Authenticated users only
- `/admin/**` → Admin users only

These rules are configured using `HttpSecurity`.

Question 11: What are `requestMatchers()`, `permitAll()`, and `authenticated()`?

Simple Answer

These methods are used in Spring Security to define which URLs are public and which require authentication.

- **`requestMatchers()`** → Selects the URLs.
- **`permitAll()`** → Allows everyone to access the URL.
- **`authenticated()`** → Allows only logged-in users.

Detailed Explanation

Spring Security allows us to define different security rules for different endpoints.

`requestMatchers()` identifies the URLs.

Then we specify who can access them.

- `permitAll()` → No login required.
- `authenticated()` → Login required.

This gives us fine-grained control over API security.

Real-Time Example

In an Employee Management System:

Public APIs

```
POST /login
POST /register
```


Protected APIs

```
GET /employees
POST /employees
DELETE /employees/10
```

Only login and registration should be public.

Code Example

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/login", "/register").permitAll()
    .anyRequest().authenticated()
)
```

Interview Tip

★ `permitAll()` means anyone can access the endpoint.

It **does not** mean the endpoint is ignored by Spring Security.

Key Points to Remember

- ☒ `requestMatchers()` selects URLs.
- ☒ `permitAll()` allows public access.
- ☒ `authenticated()` requires login.
- ☒ Commonly used inside `SecurityFilterChain`.

Quick Revision

Use **`requestMatchers()`** to select URLs, **`permitAll()`** for public access, and **`authenticated()`** for secured APIs.

Question 12: Why do we disable CSRF in REST APIs?

Simple Answer

CSRF protection is mainly required for session-based web applications. Since JWT-based REST APIs are stateless and do not use server-side sessions, CSRF is usually disabled.

Detailed Explanation

CSRF (Cross-Site Request Forgery) is an attack where a malicious website tricks a logged-in user into performing unwanted actions.

Traditional applications use:

- Session
- Cookies

REST APIs generally use:

- JWT
- Authorization Header

Since JWT is sent in the request header instead of cookies, CSRF attacks are much less applicable.

Therefore, in JWT-based applications, we usually disable CSRF.

Code Example

```
http.csrf(csrf -> csrf.disable());
```

Real-Time Example

React Application

↓

JWT stored

↓

Authorization Header

↓

Spring Boot API

Since authentication uses JWT instead of session cookies, CSRF protection is typically disabled.

Interview Tip

★ Never say:

"CSRF is useless."

Instead say:

"CSRF is important for session-based authentication but is usually disabled in stateless JWT-based REST APIs."

Key Points to Remember

-  Prevents Cross-Site Request Forgery.
-  Needed for session-based authentication.
-  Usually disabled for JWT APIs.
-  Common interview question.

Quick Revision

JWT APIs generally disable CSRF because they use stateless authentication instead of sessions.

Question 13: Why is `SessionCreationPolicy.STATELESS` used?

Simple Answer

STATELESS tells Spring Security not to create or use HTTP sessions. Every request is authenticated independently using the JWT token.

Detailed Explanation

Traditional login systems store user information in an HTTP session.

JWT authentication works differently.

Each request contains the JWT token.

Spring validates the token every time.

No session is stored on the server.

This improves scalability and suits microservices and REST APIs.

Code Example

```
http.sessionManagement(session ->
    session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
);
```

Real-Time Example

Client sends:

```
GET /employees
Authorization: Bearer eyJhbGciOiJIUzI1Ni...
```

Spring validates the JWT.

No server session is created.

Interview Tip

★ This question is almost always asked with JWT.

Remember:

JWT = Stateless

Key Points to Remember

- ☒ No HTTP Session.
- ☒ Every request carries JWT.
- ☒ Better scalability.
- ☒ Recommended for REST APIs.

Quick Revision

STATELESS means every request is authenticated using the JWT token without creating a server session.

Question 14: Why do we use BCryptPasswordEncoder?

Simple Answer

`BCryptPasswordEncoder` encrypts passwords before storing them in the database, making them much more secure than storing plain text passwords.

Detailed Explanation

Passwords should never be stored as plain text.

Instead,

Spring Security hashes passwords using BCrypt.

Advantages:

- One-way encryption (hashing)
- Salt added automatically
- Difficult to crack
- Same password generates different hashes

Code Example

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Encoding password

```
String encoded =
passwordEncoder.encode("password123");
```

Real-Time Example

Instead of storing

password123

Database stores

`$2a$10$Y1vVn...`

Even if the database is leaked, passwords remain protected.

Interview Tip

★ Interviewers often ask:

"Can we decrypt BCrypt passwords?"

Answer:

"No. BCrypt is a one-way hashing algorithm. Passwords are verified using the `matches()` method rather than decrypted."

Key Points to Remember

- ☒ Hashes passwords.
- ☒ Adds automatic salt.
- ☒ One-way encryption.
- ☒ Recommended by Spring Security.

Quick Revision

BCrypt securely hashes passwords before storing them in the database.

Question 15: What is UserDetails?

Simple Answer

`UserDetails` is a Spring Security interface that represents the authenticated user and contains information such as username, password, and roles.

Detailed Explanation

Spring Security uses the `UserDetails` interface to store user information during authentication.

It contains:

- Username
- Password
- Roles
- Account status
- Lock status
- Expiration status

Developers usually create a custom implementation using their `User` entity.

Code Example

```
public class CustomUserDetails  
implements UserDetails {  
  
}
```

Real-Time Example

Database

```
Username : murali  
Password : ****  
Role : ADMIN
```

Spring converts this information into a `UserDetails` object.

Interview Tip

★ `UserDetails` is **not** your database entity.

It is Spring Security's representation of the authenticated user.

Key Points to Remember

-  Represents logged-in user.
-  Contains username and password.
-  Stores authorities.
-  Used during authentication.

Quick Revision

`UserDetails` stores the authenticated user's information.

Question 16: What is `UserDetailsService`?

Simple Answer

UserDetailsService is an interface used by Spring Security to load user information from the database during authentication.

Detailed Explanation

When a user logs in,

Spring Security needs to retrieve user details.

It calls

```
loadUserByUsername()
```

The developer implements this interface to fetch the user from the database.

Code Example

```
@Service
public class CustomUserDetailsService
implements UserDetailsService {

}
```

Real-Time Example

User enters

```
murali
```

Spring Security calls

```
loadUserByUsername("murali")
```

The user is fetched from the database.

Interview Tip

★ One of the most frequently asked Spring Security questions.

Key Points to Remember

-  Loads user from database.
-  Called during login.
-  Developer provides implementation.
-  Returns UserDetails.

Quick Revision

UserDetailsService loads user information from the database.

Question 17: Explain `loadUserByUsername()`.

Simple Answer

`loadUserByUsername()` is the method used by Spring Security to retrieve a user from the database using the username.

Detailed Explanation

During login,

Spring Security calls

```
loadUserByUsername(username)
```

Developer:

- Searches database
- Finds user
- Converts user to `UserDetails`
- Returns the object

If the user is not found,

`UsernameNotFoundException` is thrown.

Code Example

```
@Override
public UserDetails loadUserByUsername(String username) {

    User user = repository.findByUsername(username)
        .orElseThrow(() ->
```

```
        new UsernameNotFoundException("User not  
found")) );  
  
        return new CustomUserDetails(user);  
    }  
}
```

Interview Tip

★ This method is **never called manually**.

Spring Security calls it automatically during authentication.

Key Points to Remember

-  Fetches user.
-  Returns UserDetails.
-  Throws UsernameNotFoundException.
-  Automatically called.

Quick Revision

`loadUserByUsername()` loads the user from the database during login.

Question 18: What is AuthenticationManager?

Simple Answer

`AuthenticationManager` is responsible for authenticating the user's username and password.

Detailed Explanation

When a user logs in,

the Login Controller sends the credentials to `AuthenticationManager`.

It verifies:

- Username
- Password

If valid,

authentication succeeds.

Otherwise,

an exception is thrown.

Code Example

```
authenticationManager.authenticate(  
    new UsernamePasswordAuthenticationToken(  
        username,  
        password  
    )  
);
```

Real-Time Example

Login API

↓

AuthenticationManager

↓

UserDetailsService

↓

Database

↓

Authentication Success

Interview Tip

★ AuthenticationManager **does not access the database directly**.

It delegates the work to AuthenticationProvider.

Key Points to Remember

-  Authenticates credentials.

-  Used during login.
-  Delegates authentication.
-  Returns Authentication object.

Quick Revision

AuthenticationManager validates username and password during login.

Question 19: What is AuthenticationProvider?

Simple Answer

`AuthenticationProvider` performs the actual authentication by validating the user's credentials.

Detailed Explanation

`AuthenticationManager` delegates authentication to one or more `AuthenticationProvider` implementations.

The most common implementation is:

`DaoAuthenticationProvider`

It:

- Calls `UserService`
- Loads user
- Compares password using `PasswordEncoder`
- Returns authenticated user

Real-Time Example

```
AuthenticationManager
    ↓
AuthenticationProvider
    ↓
UserService
    ↓
Database
```

Interview Tip

★ Remember:

AuthenticationManager delegates.

AuthenticationProvider performs.

Key Points to Remember

-  Performs authentication.
-  Uses UserDetailsService.
-  Uses PasswordEncoder.
-  Returns Authentication object.

Quick Revision

AuthenticationProvider performs the actual user authentication.

Question 20: How does Spring Security authenticate a user from the database?

Simple Answer

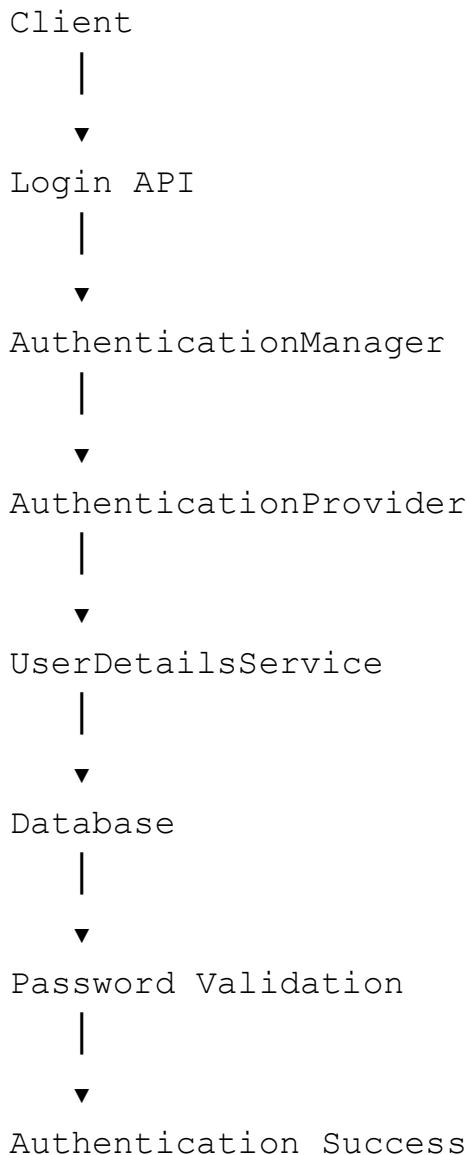
Spring Security authenticates a user by loading the user from the database using UserDetailsService, verifying the password with PasswordEncoder, and returning an authenticated Authentication object.

Detailed Explanation

Authentication Flow:

1. User submits username and password.
2. Login API calls AuthenticationManager.
3. AuthenticationManager delegates to AuthenticationProvider.
4. AuthenticationProvider calls UserDetailsService.
5. UserDetailsService loads the user from the database.
6. PasswordEncoder compares the passwords.
7. Authentication succeeds.
8. Spring creates the Authentication object.

Real-Time Example



Question 21: What is JWT?

Simple Answer

JWT (JSON Web Token) is a compact, secure, and self-contained token used to authenticate users in stateless applications. It securely transfers user information between the client and the server.

Detailed Explanation

JWT stands for **JSON Web Token**.

After a user logs in successfully:

- Spring Security verifies the username and password.
- A JWT token is generated.
- The token is returned to the client.
- The client sends this token with every request.
- Spring validates the token before processing the request.

Since the server does not store session information, JWT is ideal for REST APIs and microservices.

Real-Time Example

Employee Portal

1. User logs in.
2. Server generates a JWT.
3. Client stores the token.
4. Every API request includes the token.
5. Spring Security validates the token before allowing access.

Example JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
eyJzdWIiOiJtdXJhbGkiLCJyb2xlIjoiaURNSU4ifQ.  
abc123xyz456signature
```

Interview Tip

★ JWT is **not encryption**.

It is **digitally signed** to ensure that it has not been modified.

Key Points to Remember

-  Stateless authentication
-  Contains user information
-  Digitally signed
-  Commonly used in REST APIs

Quick Revision

JWT is a secure token used to authenticate users without storing sessions on the server.

Question 22: Why do we use JWT in REST APIs?

Simple Answer

JWT is used because REST APIs are stateless. It allows the server to authenticate every request without maintaining a session.

Detailed Explanation

REST APIs should be stateless.

This means:

- The server should not remember previous requests.
- Every request should contain all the required authentication information.

JWT solves this problem because:

- The client stores the token.
- Every request includes the token.
- The server validates the token.

No session is required.

Real-Time Example

Mobile Banking App

Instead of creating a session,

the app sends

```
Authorization: Bearer eyJhbGciOiJIUzI1Ni...
```

with every request.

Interview Tip

★ Common follow-up:

Why not use HTTP Session?

Answer:

Sessions require server memory and are less suitable for distributed systems and microservices, while JWT is stateless and scalable.

Key Points to Remember

-  Stateless
-  Scalable
-  Suitable for REST APIs
-  No server-side session

Quick Revision

JWT enables stateless authentication, making it ideal for REST APIs and microservices.

Question 23: Explain the structure of a JWT token.

Simple Answer

A JWT consists of three parts:

1. Header
2. Payload
3. Signature

These parts are separated by dots (.).

Detailed Explanation

A JWT has the following format:

Header.Payload.Signature

1. Header

Contains metadata.

Example:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload

Contains claims such as:

- Username
- Role
- Expiration time

Example:

```
{  
  "sub": "murali",  
  "role": "ADMIN"  
}
```

3. Signature

Created using:

- Header
- Payload
- Secret Key

The signature ensures the token has not been tampered with.

Real-Time Example

Header

↓

Algorithm = HS256

↓

Payload

Username = murali

Role = ADMIN

↓

Signature

Generated using Secret Key

Interview Tip

★ The payload is **Base64 encoded**, not encrypted.

Anyone can decode it, so never store sensitive data such as passwords.

Key Points to Remember

-  Header
-  Payload
-  Signature
-  Three parts separated by dots

Quick Revision

JWT consists of Header, Payload, and Signature.

Question 24: What are JWT Claims?

Simple Answer

Claims are pieces of information stored inside the JWT payload. They contain details about the user and the token.

Detailed Explanation

Claims represent information such as:

- Username
- Role
- Expiration time
- Issued time

Common standard claims include:

- `sub` → Subject (username)
- `iat` → Issued At
- `exp` → Expiration Time
- `iss` → Issuer

Custom claims can also be added.

Example:

```
{  
  "sub": "murali",
```

```
{  
  "role": "ADMIN"  
}
```

Real-Time Example

JWT Payload

```
{  
  "sub": "murali",  
  "role": "ADMIN",  
  "department": "HR"  
}
```

The application can read these claims after validating the token.

Interview Tip

★ Do not store sensitive information like passwords or credit card details in JWT claims.

Key Points to Remember

-  Claims store user information
-  Standard and custom claims
-  Stored in payload
-  Read after token validation

Quick Revision

Claims are the data stored inside the JWT payload.

Question 25: JWT vs Session-Based Authentication

Simple Answer

In session-based authentication, the server stores user sessions. In JWT authentication, the client stores the token, and the server validates it for every request.

Detailed Explanation

Session-Based Authentication

- Server stores session
- Client stores Session ID
- Requires server memory

JWT Authentication

- Client stores JWT
- Server validates JWT
- No server-side session

JWT is generally preferred for REST APIs and microservices.

Comparison

Session	JWT
Server stores session	Client stores token
Stateful	Stateless
Requires memory	No session memory
Better for traditional web apps	Better for REST APIs

Interview Tip

★ Remember:

Traditional MVC → Session

REST API → JWT

Key Points to Remember

-  Session = Stateful
-  JWT = Stateless
-  JWT improves scalability
-  JWT suits distributed systems

Quick Revision

Sessions store authentication on the server, while JWT stores authentication information in a signed token.

Question 26: Explain the complete JWT authentication flow.

Simple Answer

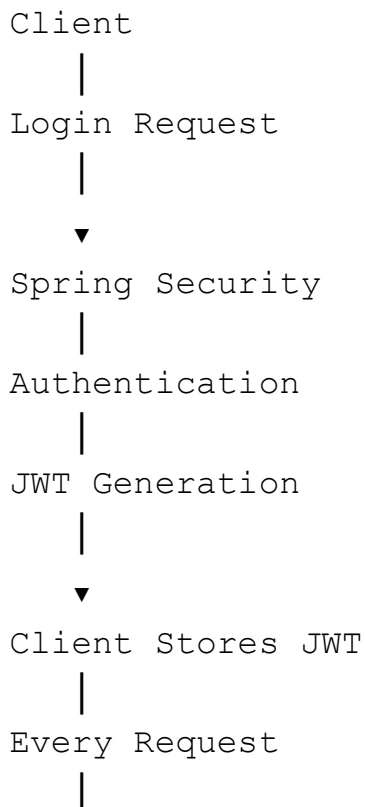
The user logs in with a username and password. After successful authentication, Spring Security generates a JWT. The client sends this JWT with every request, and Spring validates it before allowing access.

Detailed Explanation

Authentication Flow:

1. User submits username and password.
2. Spring Security authenticates the credentials.
3. JWT is generated.
4. JWT is returned to the client.
5. Client stores the JWT.
6. Every request includes the JWT.
7. Spring validates the JWT.
8. User is authenticated.

Flow Diagram



Authorization Header
|
JWT Validation
|
Controller

Interview Tip

★ This is one of the most frequently asked interview questions. Practice explaining the flow clearly in under one minute.

Key Points to Remember

-  Login
-  Authentication
-  JWT Generation
-  JWT Validation
-  Access Granted

Quick Revision

JWT authentication works by generating a token after login and validating it on every request.

Question 27: How is a JWT generated after login?

Simple Answer

After the username and password are successfully authenticated, the application creates a JWT containing user information and signs it using a secret key.

Detailed Explanation

Steps:

1. User logs in.
2. `AuthenticationManager` verifies credentials.
3. User details are loaded.
4. JWT utility creates the token.
5. Token is returned to the client.

The token typically includes:

- Username
- Roles
- Issued time
- Expiration time

Code Example

```
String token = jwtService.generateToken(userDetails);
```

Interview Tip

★ JWT generation happens **only after successful authentication**.

Key Points to Remember

- ☒ Authentication first
- ☒ Generate token
- ☒ Return token
- ☒ Client stores token

Quick Revision

JWT is generated after successful login and sent back to the client.

Question 28: How does the client send the JWT in every request?

Simple Answer

The client sends the JWT in the `Authorization` HTTP header using the Bearer scheme.

Detailed Explanation

After login, the client stores the JWT.

For every protected API request, it sends:

```
Authorization: Bearer <JWT_TOKEN>
```


Spring Security extracts the token, validates it, and authenticates the user.

Real-Time Example

```
GET /employees
```

```
Authorization: Bearer eyJhbGciOiJIUzI1Ni...
```

Interview Tip

★ The word **Bearer** must appear before the token.

Key Points to Remember

-  Uses Authorization header
-  Prefix is `Bearer`
-  Sent with every secured request
-  Validated by Spring Security

Quick Revision

Clients send JWTs using the **Authorization** header with the **Bearer** prefix.

Question 29: What is `OncePerRequestFilter`, and why is it used in JWT authentication?

Simple Answer

`OncePerRequestFilter` is a Spring Security filter that ensures the filter logic executes only once for each HTTP request. It is commonly used to validate JWT tokens.

Detailed Explanation

A custom JWT filter usually extends `OncePerRequestFilter`.

Its responsibilities are:

- Read the Authorization header.
- Extract the JWT.
- Validate the token.
- Load user details.
- Store authentication in the `SecurityContext`.

Because it executes only once per request, it avoids duplicate authentication.

Code Example

```
public class JwtAuthenticationFilter
    extends OncePerRequestFilter {

}
```

Interview Tip

★ Most JWT implementations use `OncePerRequestFilter` because it guarantees a single execution per request.

Key Points to Remember

-  Executes once per request
-  Reads JWT
-  Validates token
-  Sets authentication

Quick Revision

`OncePerRequestFilter` is used to validate JWTs once for every incoming request.

Question 30: How does Spring Security validate a JWT and authenticate the user?

Simple Answer

Spring Security extracts the JWT from the request, validates its signature and expiration, loads the user details, and creates an authenticated `Authentication` object in the `SecurityContext`.

Detailed Explanation

Validation Steps:

1. Read the Authorization header.
2. Extract the JWT.
3. Verify the signature using the secret key.
4. Check whether the token has expired.
5. Extract the username.
6. Load user details using `UserDetailsService`.
7. Create an `Authentication` object.
8. Store it in the `SecurityContext`.

If the token is invalid or expired, access is denied.

Flow Diagram

